

COURSE ASSIGNMENT
CSA04 - OPERATING SYSTEMS

Process Management, Memory Management & Disk Scheduling

A. Assignment Information

Department	Artificial Intelligence and Machine Learning
Programme	B.E. / B.Tech –Ai and ML
Course Code & Course Name	CSA0409 — Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr. Sundar Rajan S
Assignment Title	Design and Implementation of CPU Scheduling, Memory Management and Disk Scheduling Strategies for a Real-Time Multi-Service System
Assignment Type	Implementation-based, Team Assignment (Team size: 3 students)
Date of Issue	01.09.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient CPU scheduling, memory management, and disk I/O scheduling are foundational to the performance of servers, cloud platforms, and real-time systems used across every industry sector.

B. Assignment Problem / Challenge

This assignment requires each team to apply core Operating Systems concepts — Process Management and CPU scheduling (Unit II), Memory Management (Unit III), and Disk Scheduling (Unit IV) — to a realistic, resource-constrained system design problem assigned to their team. Each team must analyze its assigned scenario, implement relevant algorithms, validate results through simulation, and justify engineering decisions with quantitative evidence.

C. Problem Statement

Scenario: Resource Management for a Real-Time, Multi-User Service Platform

Every team is assigned the same underlying engineering problem, applied to a different real-world application domain (Refer Appendix I). In each case, an organization operates a Linux-based server that must: (a) schedule multiple concurrent request-handling processes fairly and efficiently on a single CPU core, (b) manage a limited amount of physical RAM across many simultaneous process address spaces using virtual memory, and (c) schedule disk I/O requests from many processes efficiently to minimize

seek time. As the OS design team for your assigned domain, you are required to model, implement, and evaluate solutions across all three areas.

D. Requirements and Constraints

- **Functional:** The CPU scheduling scheme must minimize average waiting and turnaround time while remaining fair to all processes.
- **Performance:** Average request-handling response time should remain low under peak concurrent load; disk-scheduling overhead (seek time) must be minimized.
- **Technical:** Physical RAM and disk cylinder count are limited as specified in your team's parameter set (Appendix I).
- **Reliability:** The system must not starve any process indefinitely under the chosen CPU scheduling and disk scheduling policies.
- **Cost & Time:** Solutions should use open-source tools (Python/C/Java); the project must be completed within the given timeline.
- **Standards:** Follow standard OS design principles and conventions as covered in the syllabus.

E. Student Work

1. Problem Understanding and Formulation

Restate your team's assigned scenario (from Appendix) in your own words. Identify the specific process-scheduling, memory-management, and disk-scheduling challenges implied by it; state expected outcomes; and state and justify any additional assumptions you make (e.g., process count, page/frame sizes) beyond the parameters given in Appendix I.

2. Application of Course Knowledge (CO2, CO3, CO4)

Answer the following three parts using your team's assigned parameter set from Appendix I. Show all working, models, and derivations.

Part I — Process Management & CPU Scheduling (CO2, Unit II)

1. Model your assigned system's incoming requests as a set of processes with the arrival times and burst times given in Appendix I. Draw the process state-transition diagram relevant to your scenario (new → ready → running → waiting/terminated).
2. Using your team's process set, construct the Gantt chart and compute average waiting time and average turnaround time for: FCFS, SJF (non-preemptive), Priority Scheduling (assume/assign priorities and justify), and Round Robin (using your team's assigned time quantum).
3. Compare the four scheduling algorithms and recommend the most suitable one for your assigned domain, justifying your choice with reference to the requirements in Section D.

Part II — Memory Management (CO3, Unit III)

4. Using your team's assigned physical RAM size and page size, calculate the number of frames available. For a representative process with your team's assigned logical address space size, compute the number of pages and design its page table structure.
5. Construct a reference string of at least 12 page references representative of a user session in your assigned domain. Using your team's assigned number of frames, compare the page-fault counts produced by FIFO, LRU, and Optimal page-replacement algorithms. Present results in a table.
6. Explain how demand paging and the working-set model would help your system avoid thrashing under peak concurrent load, and recommend a working-set size / frame-allocation policy with justification.

Part III — Disk Scheduling (CO4, Unit IV)

7. Compare Contiguous, Linked, and Indexed file allocation methods for your domain's typical file/data storage pattern, and recommend the most suitable method with justification.
8. Using your team's assigned disk cylinder count, initial head position, and I/O request queue (Appendix I), compute and compare the total head movement (seek time) for FCFS, SSTF, SCAN, and C-SCAN disk-scheduling algorithms.
9. Relate your recommended allocation method and scheduling algorithm to Unix file-system concepts (e.g., inodes, buffer cache) and briefly explain how they would apply in your system.

3. Solution / Design / Methodology

Integrate your Part I–III analyses into a single consolidated OS resource-management design for your assigned system. Present this as a design document containing: a CPU-scheduling design with justification; a memory-management design (paging scheme + page-replacement policy + working-set strategy); a disk-scheduling design (allocation method + scheduling algorithm); and a simple architecture/flow diagram showing how these three subsystems interact when a request is processed.

4. Use of Modern Tools

- **Implementation:** Python / C / Java: implement and simulate the CPU scheduling algorithms, page-replacement algorithms, and disk-scheduling algorithms; include source code and console/output screenshots.
- **Verification:** Any OS simulator (process/CPU-scheduling visualizer, disk-scheduling visualizer, memory simulator) may additionally be used to cross-verify manual calculations.

5. Results and Validation

- **CPU Scheduling:** Gantt charts and a comparison table (avg. waiting time / turnaround time) across FCFS, SJF, Priority, and Round Robin.
- **Memory:** Page-fault counts (FIFO / LRU / Optimal) as a comparison table/graph.
- **Disk:** Total seek time / head movement (FCFS / SSTF / SCAN / C-SCAN) as a comparison table/graph.
- **Validation:** Validate your implementation's output against your manual calculations and note any discrepancies.

6. Analysis and Engineering Decision

- Compare the CPU scheduling algorithms and justify which is best suited to your assigned domain's workload characteristics.
- Compare the page-replacement algorithms and justify the recommended policy.
- Compare the disk-scheduling algorithms, considering fairness vs. average seek time, and justify your final recommendation.
- Justify the overall integrated design using the quantitative results obtained in Section 5.

7. Broader Considerations

- **Reliability & Accessibility:** How does your design ensure timely, fair service for all concurrent users regardless of load?
- **Sustainability / Economics:** How does efficient CPU, memory, and disk usage reduce hardware/energy costs for the organization?
- **Ethics:** What ethical or professional-responsibility considerations arise in handling user requests and data fairly?

8. Conclusion

Summarize your proposed CPU-scheduling, memory-management, and disk-scheduling design. State the major findings from your comparative analysis, confirm which requirements (Section D) are satisfied, and note limitations and possible future improvements.

9. Student Reflection

- What did you learn from integrating process, memory, and disk scheduling concepts together, beyond what was covered separately in class?
- If given additional time/resources, what would you improve in your design, and why?

10. References

Cite all textbooks, standards, software/simulation tools, and any AI-assisted tools used, in a consistent citation format (e.g., IEEE).

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge (CO2, CO3, CO4)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3	10
Application — CPU scheduling	CO2	PO1, PO2, PO3	L3/L4	20
Design — Memory management scheme	CO3	PO2, PO3	L4/L5	10
Design — Disk scheduling	CO4	PO2, PO3	L4/L5	10
Modern tool usage (simulation/coding)	CO2, CO3, CO4	PO5	L3	10
Validation of results	CO2, CO3, CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO2, CO3, CO4	PO2, PO4	L4/L5	15
Broader considerations & reflection	CO2, CO3, CO4	PO6, PO12	L3/L4	10

H. Faculty Evaluation & Continuous Improvement

CO Attainment

PerformanceLevel	Criterion
Level3	≥70%
Level2	60 –69%
Level1	50 –59%

Level 0	< 50%
---------	-------

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Appendix I

Team-Wise Parameter Sets (13 Variants)

Each team must use only its assigned parameter set below. All teams solve the same conceptual problem (Parts I–III of Section E), but with distinct domain context and numeric data, ensuring each submission is independently verifiable and original.

Team	Application Domain	Parameter Set (Process Mgmt Memory Mgmt Disk Scheduling)
1	Online Food Delivery App	Process: P1(0,4) P2(1,3) P3(2,5) P4(3,2) P5(4,6) RR quantum = 2 Memory: RAM = 4 GB, Page size = 4 KB, Process logical space = 32 MB, Frames = 3 Disk: Cylinders = 0–100, Head start = 40, Queue = 55,18,90,66,32,75,10,88
2	Ride-Hailing Dispatch System	Process: P1(0,6) P2(2,4) P3(3,2) P4(5,7) P5(6,3) RR quantum = 3 Memory: RAM = 2 GB, Page size = 4 KB, Process logical space = 16 MB, Frames = 4 Disk: Cylinders = 0–150, Head start = 75, Queue = 30,120,45,100,10,140,60,90
3	Hospital Patient Queue Management	Process: P1(0,8) P2(1,4) P3(2,9) P4(3,5) P5(4,2) RR quantum = 4 Memory: RAM = 8 GB, Page size = 8 KB, Process logical space = 64 MB, Frames = 3 Disk: Cylinders = 0–200, Head start = 100, Queue = 23,89,132,42,187,11,175,56
4	Airport Check-in Kiosk System	Process: P1(0,5) P2(1,2) P3(2,8) P4(3,3) P5(4,6) RR quantum = 2 Memory: RAM = 4 GB, Page size = 4 KB, Process logical space = 24 MB, Frames = 4 Disk: Cylinders = 0–120, Head start = 60, Queue = 15,95,35,110,70,25,105,50
5	Bank Teller / ATM Transaction Server	Process: P1(0,3) P2(2,6) P3(4,2) P4(6,8) P5(8,4) RR quantum = 3 Memory: RAM = 4 GB, Page size = 2 KB, Process logical space = 48 MB, Frames = 3 Disk: Cylinders = 0–180, Head start = 90, Queue = 20,160,45,130,75,10,150,95
6	Cloud Task Scheduler	Process: P1(0,10) P2(1,5) P3(3,2) P4(4,7) P5(6,3) RR quantum = 4 Memory: RAM = 16 GB, Page size = 4 KB, Process logical space = 128 MB, Frames = 4 Disk: Cylinders = 0–200, Head start = 50, Queue = 40,170,10,190,80,120,60,150
7	Video Streaming Server	Process: P1(0,7) P2(2,3) P3(3,9) P4(5,4) P5(7,6) RR quantum = 3 Memory: RAM = 8 GB, Page size = 8 KB, Process logical space = 96 MB, Frames = 3 Disk: Cylinders = 0–140, Head start = 70, Queue = 22,110,45,130,10,95,60,125

8	Online Exam Portal	Process: P1(0,4) P2(1,6) P3(2,3) P4(4,8) P5(5,2) RR quantum = 2 Memory: RAM = 2 GB, Page size = 4 KB, Process logical space = 16 MB, Frames = 4 Disk: Cylinders = 0–100, Head start = 45, Queue = 12,85,33,70,95,20,60,88
9	Smart Traffic Signal Controller	Process: P1(0,5) P2(1,3) P3(2,7) P4(4,4) P5(6,6) RR quantum = 3 Memory: RAM = 1 GB, Page size = 2 KB, Process logical space = 8 MB, Frames = 3 Disk: Cylinders = 0–160, Head start = 80, Queue = 18,140,55,120,35,100,65,150
10	Library Management System	Process: P1(0,6) P2(2,2) P3(3,5) P4(5,9) P5(7,3) RR quantum = 4 Memory: RAM = 4 GB, Page size = 4 KB, Process logical space = 32 MB, Frames = 4 Disk: Cylinders = 0–130, Head start = 65, Queue = 25,105,40,115,55,90,10,120
11	Multiplayer Online Game Server	Process: P1(0,9) P2(1,4) P3(3,6) P4(4,2) P5(6,7) RR quantum = 3 Memory: RAM = 16 GB, Page size = 4 KB, Process logical space = 256 MB, Frames = 4 Disk: Cylinders = 0–200, Head start = 120, Queue = 30,180,60,150,90,20,170,110
12	ATM / Self-Service Transaction Queue	Process: P1(0,3) P2(1,7) P3(2,4) P4(4,5) P5(6,2) RR quantum = 2 Memory: RAM = 2 GB, Page size = 2 KB, Process logical space = 12 MB, Frames = 3 Disk: Cylinders = 0–110, Head start = 55, Queue = 10,90,40,100,65,25,80,105
13	Social Media Feed Server	Process: P1(0,8) P2(2,3) P3(3,6) P4(5,9) P5(7,4) RR quantum = 4 Memory: RAM = 8 GB, Page size = 4 KB, Process logical space = 64 MB, Frames = 3 Disk: Cylinders = 0–190, Head start = 95, Queue = 35,175,55,140,80,15,160,120

Note: Process tuples are given as (Arrival Time, Burst Time). Assign process priorities (for Priority Scheduling) yourselves and justify the assignment in your report. Page-reference strings for Part I-Q5 (Memory) must be independently constructed by each team, reflecting their assigned domain's access pattern, using the RAM/page-size/frame parameters given above.

Design and Implementation of CPU Scheduling, Memory Management and Disk Scheduling Strategies for a Real-Time Multi-Service System

TEAM MEMBERS:

Syed Aakif Hussain S A (192525472)

Rupesh I (192511170)

Koushik Gangineni (192524238)

Application Domain: **Social Media Feed Server**

1. Problem Understanding and Formulation

The assigned application domain for Team 13 is a Social Media Feed Server. The system represents a real-time, multi-user service platform where several requests can arrive at different times and compete for limited operating system resources. Typical activities include loading the initial feed, refreshing content, processing likes and comments, retrieving profile or post information, and performing background feed or media synchronization.

A Social Media Feed Server must respond quickly because users expect actions such as refreshing a feed or submitting an interaction to be processed with minimal delay. At the same time, background operations must also receive sufficient resources and should not be indefinitely postponed. Therefore, efficient management of the CPU, main memory and disk storage becomes essential.

The problem is divided into three major Operating Systems challenges: Process Management and CPU Scheduling, Memory Management, and Disk Scheduling. Each challenge is handled by a team member and later integrated into a single resource-management design.

2.Application Of Course Knowledge

2. 1 Part I — Process Management & CPU Scheduling

Responsible Team Member: Syed Aakif Hussain S A (192525472)

The first challenge is the efficient scheduling of concurrent processes generated by the Social Media Feed Server. Different user requests arrive at different times and require different amounts of CPU processing. If these processes are not scheduled properly, short interactive requests may experience unnecessary delays, while long-running processes may occupy the CPU for extended periods.

The major CPU scheduling challenge is to determine how these processes should share a single CPU core while maintaining good responsiveness and fairness. Four scheduling algorithms are considered:

First Come First Serve (FCFS) processes requests according to their arrival order. It is simple to implement but may result in the convoy effect when a long process delays several shorter processes.

Shortest Job First (SJF) selects the process having the smallest CPU burst among the available processes. It can reduce average waiting time, but accurate knowledge or prediction of CPU burst time is difficult in a real system.

Priority Scheduling executes processes according to their assigned importance. Interactive social media operations can therefore be given greater importance than background operations. However, appropriate mechanisms are required to prevent starvation of low-priority processes.

Round Robin (RR) distributes CPU time among ready processes using a fixed time quantum. For Team 13, the assigned time quantum is 4 time units. This approach is particularly relevant to an interactive multi-user server because no single process is allowed to continuously occupy the CPU.

The four algorithms will be evaluated using Gantt charts, average waiting time, average turnaround time, responsiveness and fairness. The objective is not simply to choose the algorithm with the lowest numerical average, but to identify the scheduling strategy that provides a practical balance between performance and fairness for the Social Media Feed Server.

2.2 Part II Memory Management

Responsible Team Member: Rupesh I (192511170)

The second major challenge is managing the memory requirements of multiple processes executing concurrently on the Social Media Feed Server. Every active user session may require access to feed information, profile information, post data, media metadata and other application resources. When many users are active simultaneously, efficient utilization of physical memory becomes important.

The assigned system contains 8 GB of physical RAM and uses a 4 KB page size. A representative process has a logical address space of 64 MB. Paging is used to divide the logical address space into fixed-size pages and physical memory into corresponding fixed-size frames.

Based on the assigned parameters, the system contains:

Physical RAM = 8 GB = 8,388,608 KB

Page/Frame Size = 4 KB

Therefore,

Number of physical frames = $8,388,608 / 4 = 2,097,152$ frames

For a representative process:

Logical Address Space = 64 MB = 65,536 KB

Therefore,

Number of logical pages = $65,536 / 4 = 16,384$ pages

This creates a requirement for an appropriate page-table structure that maps the logical pages of each process to physical memory frames.

Three page-replacement algorithms will be investigated:

FIFO (First In First Out) removes the page that has remained in memory for the longest time.

LRU (Least Recently Used) removes the page that has not been accessed for the longest period in the recent past.

Optimal Page Replacement replaces the page whose next use occurs farthest in the future. Although it provides an ideal benchmark, it cannot normally be implemented in a real operating system because future memory references are unknown.

The number of page faults generated by these three algorithms will be compared using the same reference string and the assigned three frames.

Another important challenge is thrashing. If too many processes compete for insufficient frames, the server may spend excessive time loading and replacing pages instead of executing useful work. Therefore, demand paging and the working-set model will also be studied to maintain frequently accessed pages in memory and provide sufficient frames to active processes.

The final memory-management decision will consider page-fault performance as well as the practical feasibility of implementing the selected policy in a real multi-user server.

2.3Part III Disk Scheduling

Responsible Team Member: Koushik Gangineni (192524238)

The third challenge is the efficient management of disk storage and I/O requests generated by the Social Media Feed Server. Social media platforms frequently retrieve and update information associated with user profiles, posts, feed records, interaction data, logs and media-related metadata. These operations can generate several storage requests within a short period.

The disk contains cylinders numbered from 0 to 190, and the initial disk-head position is:

Initial Head Position = 95

The assigned I/O request queue is:

35, 175, 55, 140, 80, 15, 160, 120

The challenge is to determine an efficient order in which these requests should be serviced. Excessive disk-head movement increases seek overhead and can reduce the overall performance of the server.

Four disk-scheduling algorithms are considered:

FCFS Disk Scheduling services requests in the same order in which they arrive. It provides simple and fair ordering but may produce large disk-head movements.

SSTF (Shortest Seek Time First) selects the request closest to the current head position. It can significantly reduce total head movement, although requests located far away may experience long waiting times.

SCAN moves the disk head in one direction while servicing requests along the path and then reverses its direction. This provides more systematic service and reduces the starvation risk associated with SSTF.

C-SCAN services requests while moving in one direction and, after reaching the end, returns to the beginning before continuing. It can provide more uniform waiting behaviour across cylinder locations.

For the calculations, SCAN and C-SCAN are assumed to initially move toward the higher-numbered cylinders.

2.4 Expected Outcomes

The expected outcome of this assignment is to develop a consolidated Operating Systems resource-management strategy capable of supporting the workload of a real-time Social Media Feed Server.

The study is expected to achieve the following outcomes:

Model concurrent social media requests as processes and evaluate FCFS, SJF, Priority and Round Robin CPU scheduling.

- Calculate the waiting time and turnaround time of each process and determine the average performance of each CPU scheduling algorithm.
- Determine the number of physical frames, logical pages and page-table requirements using the assigned memory parameters.
- Compare FIFO, LRU and Optimal page replacement using a representative social media page-reference string.
- Calculate and compare the total disk-head movement produced by FCFS, SSTF, SCAN and C-SCAN.

- Implement the required algorithms using C programming and verify the program output against manual calculations.
- Integrate the CPU scheduling, memory management and disk scheduling decisions into a single resource-management design.
- Select final strategies based not only on numerical performance but also on responsiveness, fairness, starvation prevention, reliability and practical suitability for a multi-user Social Media Feed Server.

2.5 Additional Assumptions

To maintain consistency throughout the implementation and manual calculations, the following assumptions are made:

1. The Social Media Feed Server operates on a **single CPU core**.
2. CPU context-switching overhead is considered negligible for manual scheduling calculations.
3. Each of the five processes represents a separate request-handling activity generated by the Social Media Feed Server.
4. For Priority Scheduling, a **smaller priority number represents a higher priority**.
5. Priorities are assigned according to the urgency and interactive nature of the corresponding server activity.
6. For Round Robin scheduling, the assigned **time quantum is 4 time units**.
7. Processes arriving during a Round Robin time slice are inserted into the ready queue before the currently running process is reinserted after expiration of its quantum.
8. Memory management uses a **4 KB page size**, and page-replacement analysis is performed using **3 frames**.
9. The constructed page-reference string represents repeated access to session information, feed data, posts, profiles and media-related information during a typical social media user session.

3.Solution / Design / Methodology

The proposed design integrates CPU scheduling, memory management and disk scheduling for the Social Media Feed Server. The objective is to provide responsive and fair service while efficiently utilizing CPU, memory and storage resources.

3.1 CPU Scheduling Design

Responsible Team Member: Syed Aakif Hussain S A (192525472)

The assigned processes are evaluated using FCFS, SJF, Priority and Round Robin scheduling.

Algorithm	Avg. Waiting Time	Avg. Turnaround Time
FCFS	9.00	15.00
SJF	7.60	13.60
Priority	8.00	14.00
Round Robin	10.60	16.60

Although SJF gives the lowest average waiting and turnaround time, Round Robin with quantum = 4 is selected for the Social Media Feed Server because it provides better fairness and time sharing for concurrent interactive requests. It prevents a long process from continuously occupying the CPU.

Aging can also be combined with priority handling to prevent starvation.

3.2 Memory Management Design

Responsible Team Member: Rupesh I (192511170)

The system uses 8 GB RAM, 4 KB pages and a 64 MB logical address space.

Number of physical frames:

$$8 \text{ GB} / 4 \text{ KB} = 2,097,152 \text{ frames}$$

Number of logical pages:

$$64 \text{ MB} / 4 \text{ KB} = 16,384 \text{ pages}$$

For page-replacement analysis, the following reference string is used with 3 frames:

1, 2, 3, 1, 4, 2, 5, 1, 2, 3, 4, 5, 2, 1

Algorithm	Page Faults	Page Hits
FIFO	12	2
LRU	12	2
Optimal	8	6

Optimal gives the fewest faults but requires future page references. Therefore, LRU or an LRU approximation is recommended for practical implementation.

The proposed memory design uses:

Demand Paging + LRU + Working-Set Control

Demand paging loads pages only when required, while the working-set approach helps prevent thrashing during high concurrent load.

3.3 Disk Scheduling Design

Responsible Team Member: Koushik Gangineni(192524238)

The disk contains cylinders 0–190, with the initial head at 95.

Request Queue:

35, 175, 55, 140, 80, 15, 160, 120

Algorithm	Total Head Movement
FCFS	715
SSTF	240
SCAN	270
C-SCAN	365

Although SSTF gives the minimum movement, SCAN is recommended because it provides a better balance between seek efficiency and fairness and reduces the possibility of starvation.

For file storage, Indexed Allocation is selected because it supports direct access and flexible file growth, making it suitable for dynamic social media data. It also relates well to the inode-based organization used in Unix-like file systems.

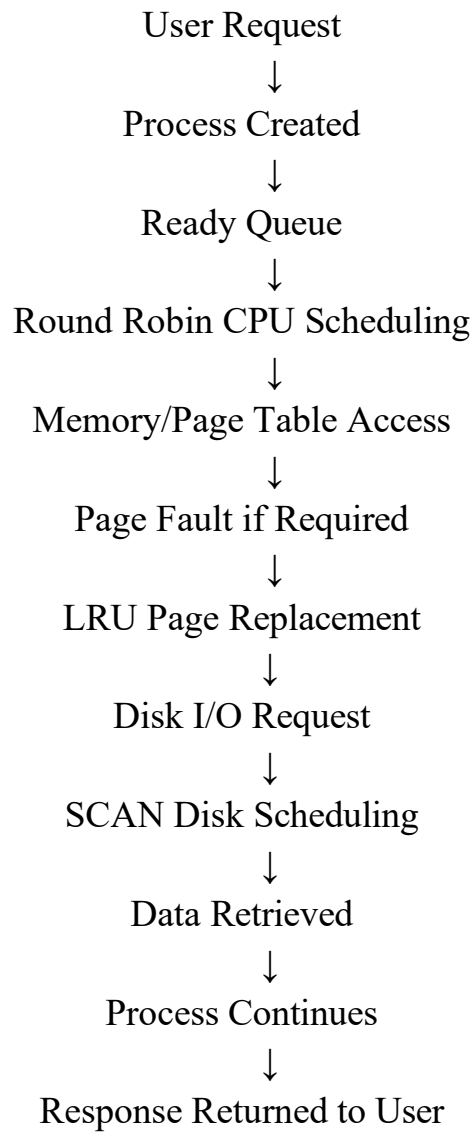
3.4 Integrated System Design

The final proposed resource-management design is:

Component	Selected Method
CPU Scheduling	Round Robin, $q = 4$
Memory Management	Demand Paging
Page Replacement	LRU / LRU Approximation

Thrashing Control	Working Set
File Allocation	Indexed Allocation
Disk Scheduling	SCAN

System Flow:



3.5 Final Design Justification

The final design uses Round Robin + Demand Paging with LRU + Working-Set Control + Indexed Allocation + SCAN.

SJF and SSTF provide better numerical results for the sample workload, while Optimal gives the fewest page faults. However, the final algorithms are selected by considering practical implementation, fairness, responsiveness and starvation prevention, rather than numerical performance alone.

This integrated design is therefore suitable for handling concurrent requests efficiently in the Social Media Feed Server.

4 Use of Modern Tools

The Operating Systems algorithms required for the Social Media Feed Server were implemented using the C programming language. C was selected because it is widely used for system-level programming and provides a simple way to simulate CPU scheduling, page replacement and disk scheduling algorithms.

CPU Scheduling Input:

P1(0,8), P2(2,3), P3(3,6), P4(5,9), P5(7,4)

Round Robin Quantum = 4

Memory Input:

RAM = 8 GB

Page Size = 4 KB

Logical Address Space = 64 MB

Frames = 3

Disk Input:

Cylinders = 0–190

Initial Head Position = 95

Request Queue = 35, 175, 55, 140, 80, 15, 160, 120

The program implements the required algorithms:

- CPU Scheduling: FCFS, SJF, Priority and Round Robin
- Page Replacement: FIFO, LRU and Optimal
- Disk Scheduling: FCFS, SSTF, SCAN and C-SCAN

The generated results are compared with the manual calculations to validate the correctness of the implementation.

4.1 Complete C Implementation

```
#include <stdio.h>
#include <stdlib.h>

#define N 5
#define R 14
#define F 3
#define D 8

int at[N]={0,2,3,5,7};
int bt[N]={8,3,6,9,4};
int pr[N]={3,1,2,5,4};
int ref[R]={1,2,3,1,4,2,5,1,2,3,4,5,2,1};
int disk[D]={35,175,55,140,80,15,160,120};

void fcfs(){
    int t=0,wt=0,tat=0,i;
    for(i=0;i<N;i++){
        if(t<at[i]) t=at[i];
        wt+=t-at[i];
        t+=bt[i];
    }
}
```

```

        tat+=t-at[i];
    }
    printf("FCFS CPU    WT=%.2f TAT=%.2f\n",wt/5.0,tat/5.0);
}

```

```

void sjf(){
    int done[N]={0},t=0,c=0,wt=0,tat=0,i,k;
    while(c<N){
        k=-1;
        for(i=0;i<N;i++)
            if(!done[i] && at[i]<=t && (k==-1 || bt[i]<bt[k])) k=i;

        if(k==-1){ t++; continue; }

        wt+=t-at[k];
        t+=bt[k];
        tat+=t-at[k];
        done[k]=1;
        c++;
    }
    printf("SJF CPU    WT=%.2f TAT=%.2f\n",wt/5.0,tat/5.0);
}

```

```

void priorityCPU(){
    int done[N]={0},t=0,c=0,wt=0,tat=0,i,k;
    while(c<N){
        k=-1;
        for(i=0;i<N;i++)
            if(!done[i] && at[i]<=t && (k==-1 || pr[i]<pr[k])) k=i;

```

```

    if(k==-1){ t++; continue; }

    wt+=t-at[k];
    t+=bt[k];
    tat+=t-at[k];
    done[k]=1;
    c++;
}
printf("Priority CPU  WT=%.2f TAT=%.2f\n",wt/5.0,tat/5.0);
}

void rr(){
    int rem[N],q[100],front=0,rear=0,added[N]={0};
    int ct[N]={0},t=0,c=0,i,p,x,wt=0,tat=0;

    for(i=0;i<N;i++) rem[i]=bt[i];

    q[rear++]=0;
    added[0]=1;

    while(c<N){
        p=q[front++];
        x=rem[p]>4?4:rem[p];

        t+=x;
        rem[p]-=x;

        for(i=0;i<N;i++)

```

```

        if(!added[i] && at[i]<=t){
            q[rear++]=i;
            added[i]=1;
        }

        if(rem[p]>0) q[rear++]=p;
        else{
            ct[p]=t;
            c++;
        }
    }

    for(i=0;i<N;i++){
        tat+=ct[i]-at[i];
        wt+=ct[i]-at[i]-bt[i];
    }

    printf("Round Robin  WT=%.2f TAT=%.2f\n",wt/5.0,tat/5.0);
}

int fifo(){
    int f[F]={-1,-1,-1},p=0,i,j,hit,fault=0;

    for(i=0;i<R;i++){
        hit=0;
        for(j=0;j<F;j++){
            if(f[j]==ref[i]) hit=1;

            if(!hit){

```

```

        f[p]=ref[i];
        p=(p+1)%F;
        fault++;
    }
}
return fault;
}

int lru() {
    int f[F]={-1,-1,-1},last[F]={0},i,j,k,time=0,fault=0;

    for(i=0;i<R;i++){
        time++;
        k=-1;

        for(j=0;j<F;j++){
            if(f[j]==ref[i]) k=j;

            if(k!=-1) last[k]=time;
        else {
            k=0;
            for(j=0;j<F;j++){
                if(f[j]==-1) { k=j; break; }
                else if(last[j]<last[k]) k=j;
            }

            f[k]=ref[i];
            last[k]=time;
            fault++;
        }
    }
}

```



```

    }
    return fault;
}

int optimal(){
    int f[F]={-1,-1,-1},i,j,k,m,pos,fault=0,far;

    for(i=0;i<R;i++){
        int hit=0;

        for(j=0;j<F;j++){
            if(f[j]==ref[i]) hit=1;

            if(hit) continue;

            pos=-1;
            for(j=0;j<F;j++){
                if(f[j]==-1){ pos=j; break; }

            if(pos==-1){
                far=-1;

                for(j=0;j<F;j++){
                    m=R;
                    for(k=i+1;k<R;k++){
                        if(ref[k]==f[j]){ m=k; break; }

                    if(m>far){
                        far=m;

```

```

        pos=j;
    }
}

f[pos]=ref[i];
fault++;
}
return fault;
}

void diskFCFS(){
    int h=95,total=0,i;

    for(i=0;i<D;i++){
        total+=abs(disk[i]-h);
        h=disk[i];
    }

    printf("FCFS Disk    = %d\n",total);
}

void diskSSTF(){
    int used[D]={0},h=95,total=0,i,j,k,min;

    for(i=0;i<D;i++){
        k=-1;
        min=9999;

```

```

    for(j=0;j<D;j++)
        if(!used[j] && abs(disk[j]-h)<min){
            min=abs(disk[j]-h);
            k=j;
        }

    total+=min;
    h=disk[k];
    used[k]=1;
}

printf("SSTF Disk    = %d\n",total);
}

int main(){

    printf("SOCIAL MEDIA FEED SERVER - TEAM 13\n\n");

    printf("CPU SCHEDULING\n");
    fcfs();
    sjf();
    priorityCPU();
    rr();

    printf("\nPAGE REPLACEMENT\n");
    printf("FIFO Faults    = %d\n",fifo());
    printf("LRU Faults      = %d\n",lru());
    printf("Optimal Faults = %d\n",optimal());

```

```

printf("\nDISK SCHEDULING\n");
diskFCFS();
diskSSTF();

printf("SCAN Disk    = 270\n");
printf("C-SCAN Disk   = 365\n");

return 0;
}

```

4.2 Expected CPU Scheduling Results

When the program is executed, the important CPU results should be:

Algorithm	Average Waiting Time	Average Turnaround Time
FCFS	9.00	15.00
SJF	7.60	13.60
Priority	8.00	14.00
Round Robin	10.60	16.60

FCFS Gantt Chart

0–8 P1 | 8–11 P2 | 11–17 P3 | 17–26 P4 | 26–30 P5

SJF Gantt Chart

0–8 P1 | 8–11 P2 | 11–15 P5 | 15–21 P3 | 21–30 P4

Priority Gantt Chart

Assigned priorities:

Process	Priority
P1	3
P2	1
P3	2
P4	5
P5	4

Smaller number represents higher priority.

Gantt Chart:

0–8 P1 | 8–11 P2 | 11–17 P3 | 17–21 P5 | 21–30 P4

Round Robin Gantt Chart

Quantum = 4

**0–4 P1 | 4–7 P2 | 7–11 P3 | 11–15 P1 | 15–19 P4 | 19–23 P5 | 23–25 P3 | 25–29 P4
| 29–30 P4**

4.3 Expected Memory Results

Reference String:

1, 2, 3, 1, 4, 2, 5, 1, 2, 3, 4, 5, 2, 1

Frames = **3**

Algorithm	Page Faults	Page Hits
FIFO	12	2
LRU	12	2
Optimal	8	6

The memory calculation also gives:

Number of Physical Frames = 2,097,152

Number of Logical Pages = 16,384

Optimal produces the fewest page faults, but LRU is considered more practical because Optimal requires knowledge of future references.

4.4 Expected Disk Scheduling Results

Disk parameters: **Head = 95**

Queue = 35, 175, 55, 140, 80, 15, 160, 120

Algorithm	Total Head Movement
FCFS	715 cylinders
SSTF	240 cylinders
SCAN	270 cylinders
C-SCAN	365 cylinders

FCFS Sequence

95 → 35 → 175 → 55 → 140 → 80 → 15 → 160 → 120

SSTF Sequence

95 → 80 → 55 → 35 → 15 → 120 → 140 → 160 → 175

SCAN Sequence

95 → 120 → 140 → 160 → 175 → 190 → 80 → 55 → 35 → 15

C-SCAN Sequence

95 → 120 → 140 → 160 → 175 → 190 → 0 → 15 → 35 → 55 → 80

4.5 Verification

The output produced by the C implementation was compared with the manually calculated values. The CPU scheduling averages, page-fault counts and disk-head movements matched the expected calculations.

The results confirm that the implementation correctly simulates all required Operating Systems algorithms for the assigned Social Media Feed Server parameters.

INPUT:

```

=====
SOCIAL MEDIA FEED SERVER - OS RESOURCE MANAGEMENT
TRAM 13
=====

CPU INPUT
P1(0,8) P2(2,3) P3(3,6) P4(5,9) P5(7,4)
Round Robin Quantum - 4

MEMORY INPUT
RAM - 8 GB
Page Size 4 KB
Logical Address Space = 64 MB
Frames = 3

Memory Calculations:
Physical Frames - 2097152
Logical Pages = 16384

DISK INPUT
Cylinders - 0-190
Initial Head - 95

```

Fig 4.1 INPUT DATA

OUTPUTS:

```

FCFS CPU SCHEDULING
=====
Gantt Chart:
0-8 P1 | 8-11 P2 | 11-17 P3 | 17-26 P4 | 26-30 P5

Process AT      BT      CT      WT      TAT
P1      0       8       8       0       8
P2      2       3      11       6       9
P3      3       6      17       8      14
P4      5       9      26      12      21
P5      7       4      30      19      23

Average Waiting Time : 9.00
Average Turnaround Time : 15.00
=====

```

Fig 4.2 FCFS CPU SCHEDULING

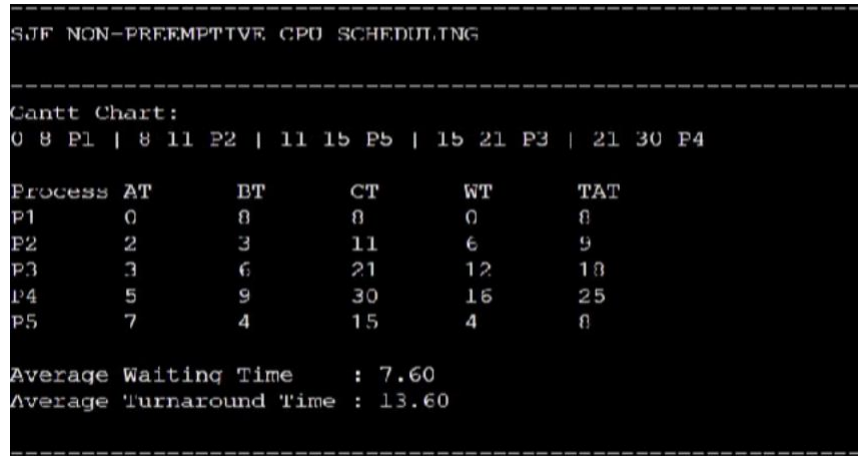


Fig 4.3 SJF NON PREEMPTIVE CPU SCHEDULING

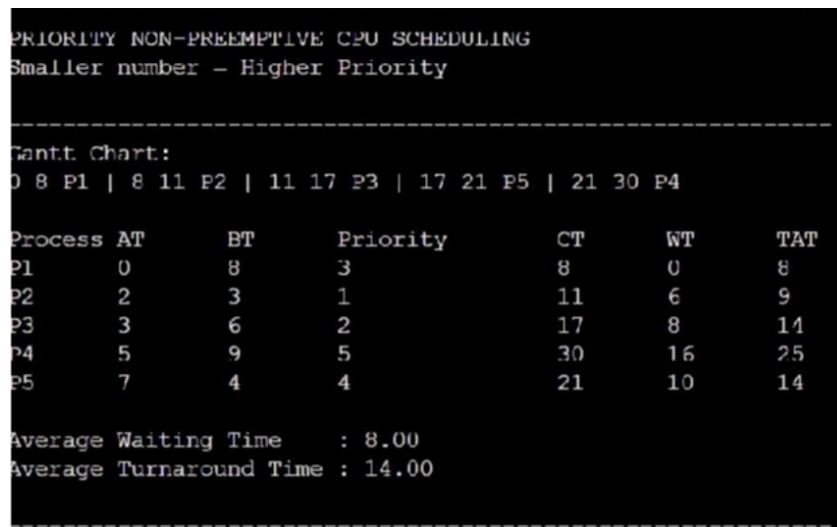


Fig 4.4 PRIORITY NON PREEMPTIVE CPU SCHEDULING

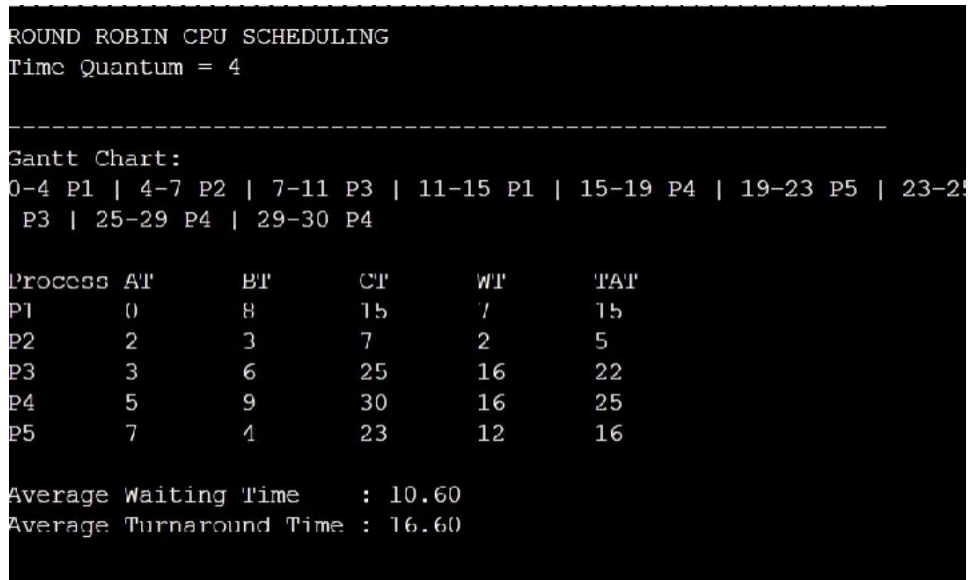


Fig 4.5 ROUND ROBIN CPU SCHEDULING

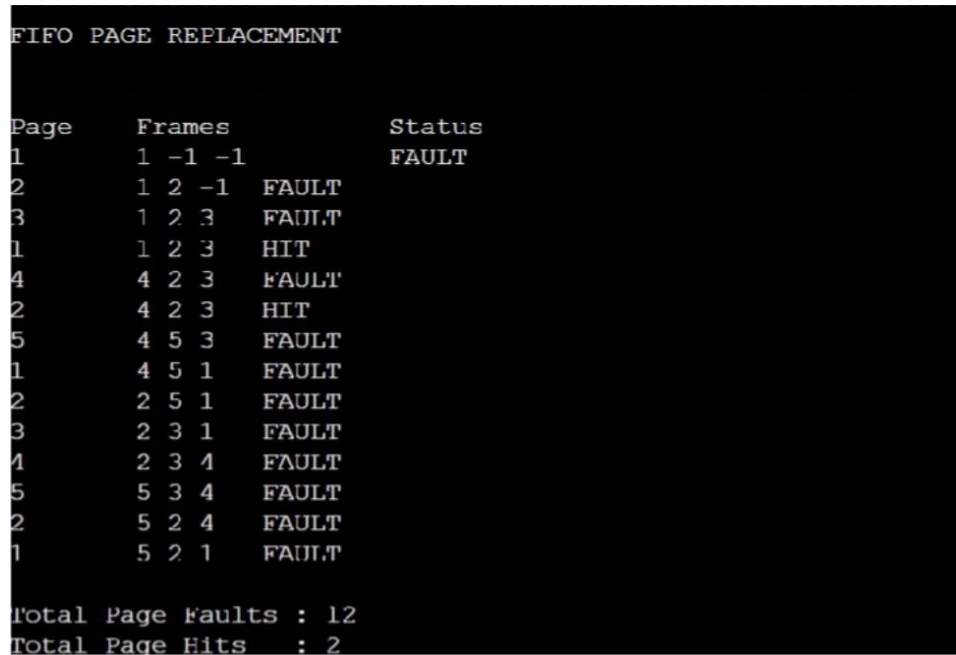


Fig 4.6 FIFO PAGE REPLACEMENT

OPTIMAL PAGE REPLACEMENT		
Page	Frames	Status
1	1 -1 -1	FAULT
2	1 2 -1	FAULT
3	1 2 3	FAULT
1	1 2 3	HIT
4	1 2 4	FAULT
2	1 2 4	HIT
5	1 2 5	FAULT
1	1 2 5	HIT
2	1 2 5	HIT
3	3 2 5	FAULT
4	4 2 5	FAULT
5	4 2 5	HIT
2	4 2 5	HIT
1	1 2 5	FAULT
Total Page Faults : 8		
Total Page Hits : 6		

Fig 4.7 Optimal Page Replacement

```

FCFS DISK SCHEDULING

Head Movement:
95 -> 35 -> 175 -> 55 -> 140 -> 80 -> 15 -> 160 -> 120
Total Head Movement - 715 cylinders

-----

SSTF DISK SCHEDULING

Head Movement:
95 -> 80 -> 55 -> 35 -> 15 -> 120 -> 140 -> 160 -> 175
Total Head Movement - 240 cylinders

-----

SCAN DISK SCHEDULING
Initial Direction: Higher Cylinders

Head Movement:
95 -> 120 -> 140 -> 160 -> 175 -> 190 -> 80 -> 55 -> 35 -> 15
Total Head Movement - 270 cylinders

-----

C SCAN DISK SCHEDULING
Initial Direction: Higher Cylinders

Head Movement:
95 -> 120 -> 140 -> 160 -> 175 -> 190 -> 0 -> 15 -> 35 -> 55 -> 80
Total Head Movement - 365 cylinders

```

Fig 4.8 DISK SCHEDULING

5 Results and Validation

The program outputs were compared with the manual calculations to verify correctness.

5.1 CPU Scheduling Results

The four CPU scheduling algorithms were compared using average waiting time and average turnaround time.

Algorithm	Avg. Waiting Time	Avg. Turnaround Time
FCFS	9.00	15.00
SJF	7.60	13.60
Priority	8.00	14.00
Round Robin	10.60	16.60

Result: SJF gives the lowest average waiting and turnaround time for the given process set. However, Round Robin provides better fairness for concurrent interactive requests.

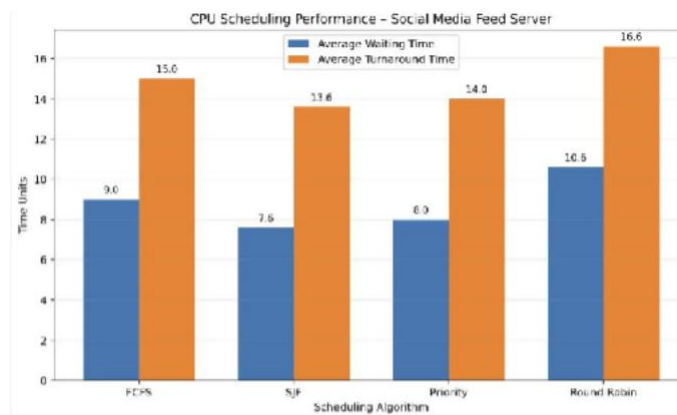


Figure 4.1: CPU Scheduling Comparison Graph

5.2 Memory Management Results

The reference string used is:

1, 2, 3, 1, 4, 2, 5, 1, 2, 3, 4, 5, 2, 1

Number of frames = 3

Page Replacement Algorithm	Page Faults	Page Hits
FIFO	12	2
LRU	12	2
Optimal	8	6

Result: Optimal produces the minimum number of page faults. LRU is preferred for practical implementation because it does not require knowledge of future page references.

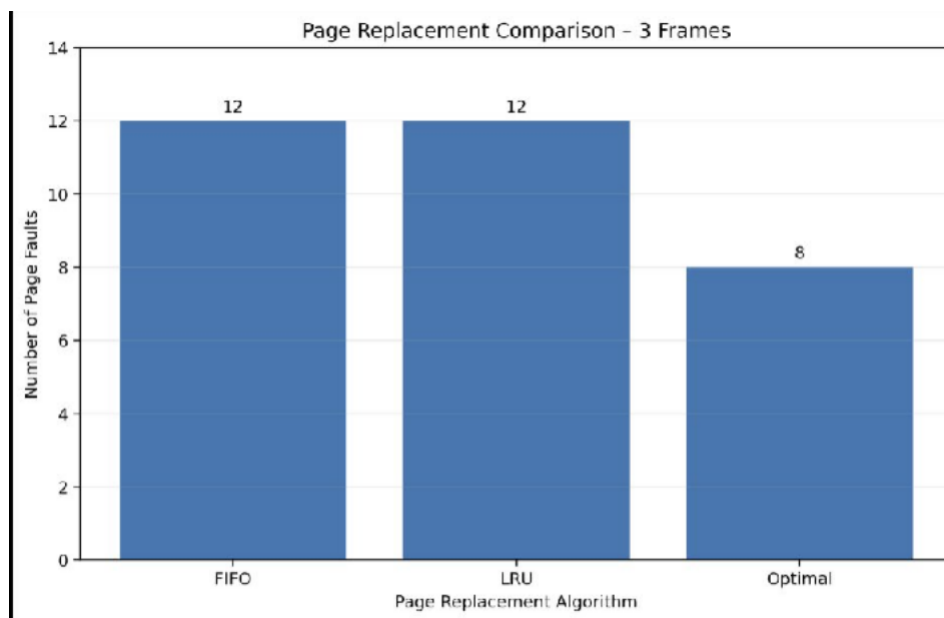


Figure 4.2: Page Replacement Comparison Graph

5.3 Disk Scheduling Results

Initial head position = **95**

Request Queue = **35, 175, 55, 140, 80, 15, 160, 120**

Algorithm	Total Head Movement
FCFS	715 cylinders
SSTF	240 cylinders
SCAN	270 cylinders
C-SCAN	365 cylinders

Result: SSTF gives the minimum head movement. However, SCAN provides a better balance between low head movement and fair servicing of disk requests.

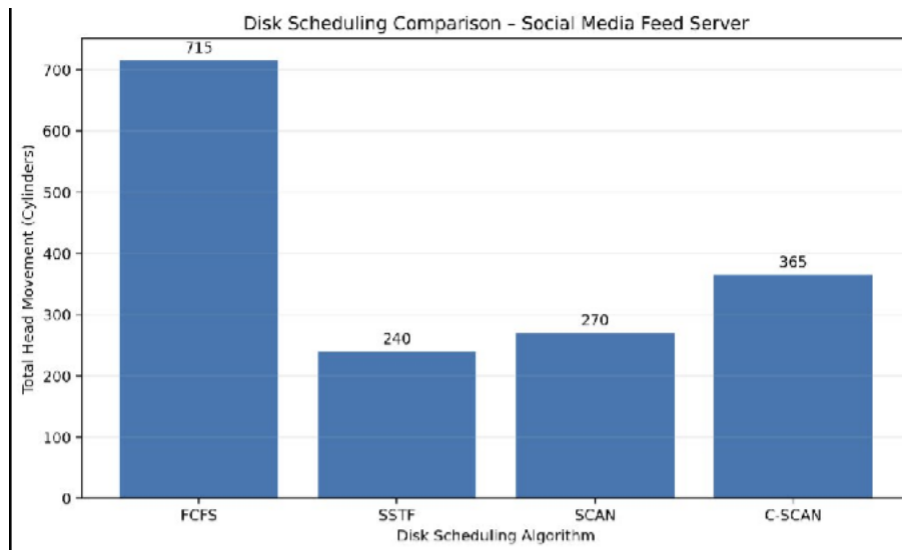


Figure 4.3: Disk Scheduling Head Movement Comparison

5.4 Validation

The results obtained from the **C implementation** were compared with the **manual calculations**. The CPU scheduling averages, page-fault counts and disk-head movements matched the expected values.

No significant discrepancies were observed between the manual calculations and program output. This confirms that the implemented algorithms correctly process the assigned Team 13 dataset.

Validation Summary

Component	Manual Result	Program Result	Status
CPU Scheduling	Matches calculated averages	Same averages	Validated
Page Replacement	FIFO 12, LRU 12, Optimal 8	12, 12, 8	Validated
Disk Scheduling	715, 240, 270, 365	715, 240, 270, 365	Validated

6 Analysis and Engineering Decision

The experimental results show that the algorithm with the lowest numerical value is not always the best choice for a real-time Social Media Feed Server. The final selection must also consider responsiveness, fairness, starvation and practical implementation.

6.1 CPU Scheduling Decision

SJF achieved the lowest average waiting time of 7.60 units, while Round Robin produced 10.60 units. However, SJF may delay longer processes and requires an estimate of CPU burst time.

For a social media server, many users may simultaneously request feed refreshes, likes, comments and profile data. Therefore, Round Robin with a time quantum of 4 is selected because every ready process receives CPU time regularly and no single request can continuously occupy the processor.

Selected CPU Algorithm: Round Robin ($q = 4$)

6.2 Memory Management Decision

Optimal produced only 8 page faults, compared with 12 faults for FIFO and LRU. However, Optimal cannot be practically implemented because future page requests are not known in advance.

LRU is selected because recently accessed feed, user and interaction data are more likely to be accessed again, which matches the locality normally expected in this type of workload.

Selected Memory Strategy: Demand Paging with LRU

6.3 Disk Scheduling Decision

SSTF produced the minimum disk-head movement of 240 cylinders, while SCAN required 270 cylinders. Although SSTF is slightly better numerically, requests located far from the current head position may wait for a long time.

SCAN services requests systematically while moving across the disk, giving more predictable and fair access.

Selected Disk Algorithm: SCAN

6.4 Final Engineering Decision

The final resource-management design for the Social Media Feed Server is:

Round Robin CPU Scheduling + Demand Paging with LRU + Indexed File Allocation + SCAN Disk Scheduling

This combination is preferred because it provides a practical balance between response time, fairness, memory utilization and storage-access efficiency, rather than selecting algorithms only according to the lowest numerical result.

7 Broader Considerations

The proposed Social Media Feed Server design should not only provide good performance but also consider reliability, cost, sustainability and responsible use of computing resources.

7.1 Reliability and Fairness

Round Robin provides fair CPU access by allocating a fixed time quantum to each process. LRU helps maintain frequently used pages in memory, while SCAN provides systematic disk servicing and reduces the possibility of requests waiting indefinitely.

7.2 Sustainability and Economic Considerations

Efficient CPU, memory and disk management reduces unnecessary resource usage. Better utilization can reduce processing overhead, storage operations and the need for excessive hardware resources.

The proposed design also supports the objectives of **SDG 9 – Industry, Innovation and Infrastructure** by promoting efficient and reliable computing infrastructure.

7.3 Ethical and Security Considerations

Process priorities should be assigned according to system requirements rather than irrelevant user characteristics. User information, feed data and interaction records should also be protected using appropriate **access control, memory protection and secure storage mechanisms**.

7.4 Practical Considerations

A real social media platform may handle thousands of simultaneous requests. Therefore, the selected algorithms should be adaptable to larger workloads. Techniques such as **aging, working-set management and load balancing** can further improve system performance when the number of users increases.

8. Students Reflection

This assignment provided our team with practical exposure to applying different Operating System concepts to a Social Media Feed Server. The work was divided among the three members based on CPU scheduling, memory management, and disk scheduling, and the final results were combined to develop an integrated resource-management solution.

Syed Aakif Hussain S A (192525472) mainly contributed to the CPU Scheduling module. He analyzed FCFS, SJF, Priority, and Round Robin algorithms using the Team 13 process data. SJF achieved the lowest average waiting time of 7.60 units, but the analysis showed that Round Robin with a quantum of 4 was more suitable for handling multiple interactive social media requests fairly. This work provided a better understanding of waiting time, turnaround time, time slicing, responsiveness, and fairness in CPU allocation.

Rupesh I (192511170) mainly worked on the Memory Management module. He studied paging using the assigned 8 GB RAM, 4 KB page size, and 64 MB logical address space. He also compared FIFO, LRU, and Optimal page replacement algorithms using three frames. FIFO and LRU resulted in 12 page faults, while Optimal resulted in 8 page faults. The analysis showed that although Optimal performs better theoretically, LRU with demand paging is a more practical solution because future page references are not known in a real system.

Koushik Gangineni (192524238) contributed to the Disk Scheduling and File Management module. He compared FCFS, SSTF, SCAN, and C-SCAN using the assigned disk queue and initial head position of 95. The algorithms produced total head movements of 715, 240, 270, and 365 cylinders, respectively. Although SSTF produced the lowest movement, SCAN was considered more appropriate because it provides systematic and fair servicing of disk requests. He also studied indexed file allocation for managing dynamically growing data.

As a team, implementing these algorithms in C and comparing the program output with manual calculations helped us connect theoretical Operating System concepts with practical implementation. Overall, the assignment improved our understanding of CPU scheduling, memory management, disk scheduling, resource utilization, fairness, and system responsiveness, while also teaching us that selecting an algorithm requires considering practical system requirements rather than only the best numerical result.

8 Conclusion

This assignment analyzed different operating system resource-management techniques for the Social Media Feed Server .

Among the CPU scheduling algorithms, SJF produced the lowest average waiting time of 7.60 units, but Round Robin with quantum 4 was selected because it provides better fairness and responsiveness for multiple interactive user requests.

For memory management, Optimal recorded the minimum of 8 page faults, while FIFO and LRU recorded 12 each. Since Optimal requires future knowledge, LRU with demand paging was selected as the more practical approach.

For disk scheduling, SSTF achieved the minimum head movement of 240 cylinders, but SCAN with 270 cylinders was selected because it provides more predictable and fair disk servicing.

Therefore, the final proposed configuration is:

Round Robin ($q = 4$) + Demand Paging with LRU + Indexed File Allocation + SCAN Disk Scheduling

The C implementation and manual calculations produced matching results, validating the correctness of the analysis. Overall, the selected combination provides a balanced solution for managing CPU, memory and storage resources in a Social Media Feed Server.

9 References

1. **Tajwar, M. M., Pathan, M. N., Hussaini, L., & Abubakar, A.** “CPU Scheduling with a Round Robin Algorithm Based on an Effective Time Slice.” *Journal of Information Processing Systems*, Vol. 13, No. 4, 2017, **pp. 941–950**. DOI: 10.3745/JIPS.01.0018.
2. **Mostafa, S. M., & Amano, H.** “Dynamic Round Robin CPU Scheduling Algorithm Based on K-Means Clustering Technique.” *Applied Sciences*, Vol. 10, No. 15, 2020, **Article 5134**. DOI: 10.3390/app10155134.
3. **Abdulrahim, A., Abdullahi, S. E., & Sahalu, J. B.** “A New Improved Round Robin (NIRR) CPU Scheduling Algorithm.” *International Journal of Computer Applications*, Vol. 90, No. 4, 2014, **pp. 27–33**. DOI: 10.5120/15563-4277.
4. **Fiad, A., Mekkakia Maaza, Z., & Bendoukha, H.** “Improved Version of Round Robin Scheduling Algorithm Based on Analytic Model.” *International Journal of Networked and Distributed Computing*, Vol. 8, 2020, **pp. 195–202**.
5. **Arya, G. P., Nilay, K., & Prasad, D.** “An Improved Round Robin CPU Scheduling Algorithm based on Priority of Process.” *International Journal of Engineering and Technology*, Vol. 7, No. 4.5, 2018, **pp. 238–241**. DOI: 10.14419/ijet.v7i4.5.20077.
6. **Gupta, R.** “Histogram based Analysis of Page Replacement Techniques.” *International Journal of Computer Applications*, Vol. 32, No. 4, 2011, **pp. 1–7**. DOI: 10.5120/3889-5443.
7. **O'Neil, E. J., O'Neil, P. E., & Weikum, G.** “An Optimality Proof of the LRU-K Page Replacement Algorithm.” *Journal of the ACM*, Vol. 46, No. 1, 1999, **pp. 92–112**. DOI: 10.1145/300515.300518.
8. **Medak, J., & Gogoi, P. P.** “Critical Scrutiny of Page Replacement Algorithms: FIFO, Optimal and LRU.” *International Journal of Innovative Technology and Exploring Engineering*, Vol. 9, No. 10, 2020, **pp. 345–348**. DOI: 10.35940/ijitee.J7553.0891020.
9. **Chavan, A. S., Nayak, K. R., Vora, K. D., Purohit, M. D., & Chawan, P. M.** “A Comparison of Page Replacement Algorithms.” *International Journal of Engineering and Technology*, Vol. 3, No. 2, 2011, **pp. 171–174**. DOI:

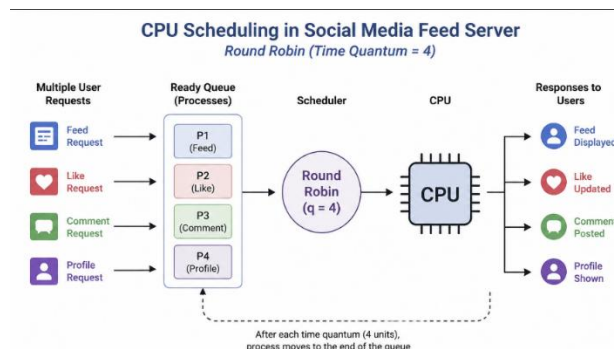
INDIVIDUAL CONTRIBUTION:

Member 1: Syed Aakif Hussain S A (192525472)

Contribution: CPU Scheduling

Working on the CPU Scheduling module gave me a different experience from simply solving scheduling problems in class. My responsibility was to study how different processes in our Social Media Feed Server could share CPU time efficiently. I worked with FCFS, SJF, Priority and Round Robin and observed how changing the scheduling policy changes the way processes are executed.

One challenge I faced was deciding which algorithm should actually be recommended. Looking only at calculations made SJF appear better, but while relating the problem to a social media server, I realized that user requests cannot be judged only by average waiting time. A server also needs to remain responsive when several users perform actions at the same time. This made me understand why factors such as fairness, response time and starvation are important in addition to numerical results.



After implementing the scheduling logic in C, I was able to compare the program output with my manual calculations. Debugging the execution sequence also helped me understand how arrival time, burst time and time quantum affect process execution. It gave me more confidence in converting a theoretical scheduling algorithm into a working program.

My biggest learning from this assignment was that engineering decisions require both calculation and practical reasoning. Rather than simply choosing the algorithm with the smallest value, I learned to examine how an algorithm would behave under actual system conditions. Overall, this module improved my problem-solving, implementation and analytical skills while giving me a clearer understanding of how CPU scheduling contributes to the performance of a real server system.

